

互斥算法专题整理

一、总体定位

互斥算法解决的是并发进程访问同一临界资源时“不能同时进入临界区”的问题。经典互斥算法通常围绕下面三个要求展开：

1. 空闲让进：临界区空闲时，申请进入的进程应能进入。
2. 忙则等待：已有进程在临界区时，其他申请者必须等待。
3. 有限等待：申请进入临界区的进程不能无限期等待。

有的教材还强调“让权等待”，即进程不能进入临界区时应释放处理机。软件互斥算法通常采用忙等方式，因此一般不能满足让权等待，但它们非常适合用来考查互斥思想、算法缺陷和并发执行序列分析。

下面按四类整理：

1. 单标志法、双标志法：常用于说明错误互斥算法的缺陷。
2. Dekker 算法和 Peterson 算法：两个进程互斥的经典软件算法。
3. Lamport 面包店算法：解决 n 个进程互斥。
4. Eisenberg/McGuire 算法：使用状态数组和 turn 指针解决 n 个进程互斥。

二、单标志法与双标志法

1. 单标志法

单标志法又常称为“严格轮换法”。它只使用一个公共变量 `turn` 指示当前允许哪个进程进入临界区。

基本实现

假设有两个进程 `P0`、`P1`，共享变量：

```
int turn = 0;
```

进程 `Pi` 的算法如下，其中 $j = 1 - i$ ：

```
while (true) {  
    while (turn != i) {  
        ; // 忙等  
    }  
  
    临界区;  
}
```

```
    turn = j;

    剩余区;
}
```

算法解释

turn 表示“轮到谁进入”。如果 `turn == 0`，只有 P0 可以进入临界区；如果 `turn == 1`，只有 P1 可以进入临界区。进程离开临界区时，把 turn 交给另一个进程。

它能够保证互斥，因为任一时刻 turn 只能等于一个进程号，不可能同时等于 0 和 1。

但是它的问题也很明显：它要求两个进程严格交替进入临界区。如果 P0 想连续进入两次，而 P1 此时一直不想进入，P0 第一次退出后把 turn 设为 1，就会被迫等待 P1。这时临界区明明是空的，却不允许 P0 进入，违反“空闲让进”。

主要缺陷

1. 保证互斥。
2. 不满足空闲让进。
3. 进程必须严格轮流，不能适应一个进程多次连续申请、另一个进程长期不申请的情况。
4. 使用忙等，占用 CPU。

2. 双标志先检查法

双标志法使用数组 `flag[2]` 表示两个进程是否想进入临界区。先检查法的思想是：先看对方是否想进入，如果对方不想进入，自己再表明意愿并进入。

基本实现

共享变量：

```
boolean flag[2] = {false, false};
```

进程 P_i 的算法如下，其中 $j = 1 - i$ ：

```
while (true) {
    while (flag[j]) {
        ; // 忙等
    }

    flag[i] = true;

    临界区;

    flag[i] = false;
```

```
    剩余区;  
}
```

算法解释

进程进入临界区前，先检查对方的 `flag[j]`。如果对方没有申请进入，就把自己的 `flag[i]` 设为 `true`，然后进入临界区。

这个算法的问题在于“检查”和“设置”不是一个不可分割的原子操作。可能出现如下交错：

```
P0 检查 flag[1] == false, 通过检查;  
P1 检查 flag[0] == false, 通过检查;  
P0 设置 flag[0] = true, 进入临界区;  
P1 设置 flag[1] = true, 进入临界区;
```

结果是 P0 和 P1 同时进入临界区，违反互斥。

主要缺陷

1. 不保证互斥。
2. 根本原因是先检查后设置，检查结果可能在设置前失效。
3. 说明仅有“意愿标志”还不够，还必须处理并发检查与并发设置的竞态问题。

3. 双标志后检查法

后检查法的思想是：先声明自己想进入，再检查对方是否也想进入。

基本实现

共享变量：

```
boolean flag[2] = {false, false};
```

进程 P_i 的算法如下，其中 $j = 1 - i$ ：

```
while (true) {  
    flag[i] = true;  
  
    while (flag[j]) {  
        ; // 忙等  
    }  
  
    临界区;  
  
    flag[i] = false;
```

```
    剩余区;  
}
```

算法解释

进程先把自己的标志设为 `true`，表示自己想进入临界区。然后检查对方是否也想进入。如果对方不想进入，自己进入临界区；如果对方也想进入，自己等待。

它能避免双标志先检查法中“两者都先通过检查”的问题，因此可以保证互斥。但是它可能导致两个进程互相等待：

```
P0 设置 flag[0] = true;  
P1 设置 flag[1] = true;  
P0 检查 flag[1] == true, 于是等待;  
P1 检查 flag[0] == true, 于是等待;
```

两个进程都在等待对方把标志改回 `false`，但它们都无法继续执行到退出临界区的位置，因而发生死锁。

主要缺陷

1. 保证互斥。
2. 可能死锁。
3. 违反空闲让进和有限等待。
4. 根本原因是两个进程都“先占位”，但没有冲突时的让步规则。

4. 本类算法例题

题目

有两个进程 `P0`、`P1`，采用如下算法实现互斥：

```
boolean flag[2] = {false, false};  
  
Process Pi:  
while (true) {  
    flag[i] = true;  
    while (flag[1 - i]) {  
        ;  
    }  
    临界区;  
    flag[i] = false;  
    剩余区;  
}
```

问：该算法能否保证互斥？是否可能发生死锁？请给出一个执行序列说明。

解析

该算法是双标志后检查法。它能保证互斥，因为一个进程若能进入临界区，说明它观察到对方的 `flag` 为 `false`；而对方若也想进入，必须先把自己的 `flag` 设为 `true`，此时会被正在进入的进程阻挡。

但是它可能发生死锁。执行序列如下：

```
P0: flag[0] = true;
P1: flag[1] = true;
P0: 检查 flag[1] == true, 进入 while 循环等待;
P1: 检查 flag[0] == true, 进入 while 循环等待;
```

此后两个进程都无法进入临界区，也无法把自己的 `flag` 改回 `false`，所以发生死锁。

三、Dekker 算法与 Peterson 算法

Dekker 算法和 Peterson 算法都解决两个进程互斥问题。它们的共同思想是把“是否想进入临界区”的标志变量和“发生冲突时谁优先”的 `turn` 变量结合起来。

1. Dekker 算法

Dekker 算法是早期经典的纯软件互斥算法。它使用两个意愿标志 `flag[2]` 和一个让步变量 `turn`。

基本实现

共享变量：

```
boolean flag[2] = {false, false};
int turn = 0;
```

进程 P_i 的算法如下，其中 $j = 1 - i$ ：

```
while (true) {
    flag[i] = true;

    while (flag[j]) {
        if (turn == j) {
            flag[i] = false;

            while (turn == j) {
                ; // 等待对方使用完优先权
            }
        }
    }
}
```

```
        flag[i] = true;
    }
}

临界区;

turn = j;
flag[i] = false;

剩余区;
}
```

算法解释

`flag[i] = true` 表示 P_i 想进入临界区。如果发现 P_j 也想进入，即 `flag[j] == true`，则发生竞争。

这时检查 `turn`：

1. 如果 `turn == j`，说明当前冲突中应让 P_j 优先。 P_i 临时撤销自己的申请，即 `flag[i] = false`，等待 `turn` 不再等于 j 后重新申请。
2. 如果 `turn != j`，说明当前不需要让给 P_j ， P_i 继续等待对方的 `flag[j]` 变为 `false` 或自己获得进入机会。

退出临界区时，进程把 `turn` 交给对方，并清除自己的意愿标志。这样做既防止两个进程同时进入，又避免双方都坚持不让导致死锁。

性质总结

1. 互斥性：可以保证两个进程不会同时进入临界区。
2. 空闲让进：若对方不想进入，自己可以直接进入。
3. 有限等待：冲突时通过 `turn` 交替让权，避免长期饥饿。
4. 缺点：算法结构较复杂，仍然是忙等算法。

2. Peterson 算法

Peterson 算法比 Dekker 算法更简洁，是考试中最常见的两个进程软件互斥算法之一。

基本实现

共享变量：

```
boolean flag[2] = {false, false};
int turn = 0;
```

进程 P_i 的算法如下，其中 $j = 1 - i$ ：

```
while (true) {
    flag[i] = true;
    turn = j;

    while (flag[j] && turn == j) {
        ; // 忙等
    }

    临界区;

    flag[i] = false;

    剩余区;
}
```

算法解释

`flag[i] = true` 表示 P_i 想进入临界区。随后 `turn = j` 表示如果双方同时竞争，自己愿意让对方先进入。

等待条件是：

```
flag[j] && turn == j
```

含义是：对方也想进入，并且当前优先权给了对方。只有这两个条件同时成立， P_i 才等待。只要对方不想进入，或者优先权不在对方， P_i 就可以进入临界区。

当两个进程几乎同时申请时，二者都会设置自己的 `flag`，并都把 `turn` 让给对方。但 `turn` 最终只能保留最后一次写入的值。若最后 `turn == 1`，则 P_0 等待、 P_1 进入；若最后 `turn == 0`，则 P_1 等待、 P_0 进入。

性质总结

1. 互斥性：可以保证两个进程不会同时进入临界区。
2. 空闲让进：若另一个进程不想进入，当前进程不会等待。
3. 有限等待：两个进程竞争时，`turn` 的让步机制避免无限等待。
4. 缺点：只适用于两个进程；属于忙等算法。

3. Dekker 与 Peterson 的比较

比较项	Dekker 算法	Peterson 算法
适用对象	两个进程	两个进程
共享变量	<code>flag[2]</code> 、 <code>turn</code>	<code>flag[2]</code> 、 <code>turn</code>
冲突处理	优先权在对方时主动撤销申请	主动把优先权让给对方

比较项	Dekker 算法	Peterson 算法
代码复杂度	较复杂	较简洁
考试重点	理解让步思想和有限等待	会写伪代码、会分析互斥性

4. 本类算法例题

题目

判断 Peterson 算法是否可能出现两个进程同时进入临界区。若不可能，请用 flag 和 turn 说明原因。

解析

不可能。

假设 P0 和 P1 同时在临界区，则它们都通过了等待条件。

P0 能进入，说明：

```
!(flag[1] && turn == 1)
```

P1 能进入，说明：

```
!(flag[0] && turn == 0)
```

如果两个进程都在临界区，那么在进入前它们都执行过：

```
flag[0] = true;
flag[1] = true;
```

并且在退出前不会把自己的 flag 改回 false。所以两个进程同时在临界区时，应有：

```
flag[0] == true 且 flag[1] == true
```

于是 P0 能进入只能说明 $turn \neq 1$ ，即 $turn == 0$ ；P1 能进入只能说明 $turn \neq 0$ ，即 $turn == 1$ 。这要求 turn 同时等于 0 和 1，矛盾。因此 Peterson 算法不会让两个进程同时进入临界区。

四、Lamport 面包店算法

Lamport 面包店算法用于解决 n 个进程的互斥问题。它模仿面包店排队取号：每个想进入临界区的进程先取一个号码，号码小者先进入；号码相同则进程号小者先进入。

注意：面包店算法是软件互斥算法，不是信号量算法。它经常和信号量、PV 操作放在同一章考查，但本质上不依赖 P、V 原语。

1. 算法思想

每个进程 P_i 有两个共享变量：

1. `choosing[i]`：表示 P_i 是否正在取号。
2. `number[i]`：表示 P_i 取到的号码。 `number[i] == 0` 表示 P_i 不想进入临界区。

进入临界区前，进程先选取一个比当前所有号码都大的号码：

```
number[i] = 1 + max(number[0], number[1], ..., number[n-1])
```

然后依次检查其他进程。若存在某个进程 P_j 已经取号，且它的优先级高于自己，则等待。

优先级按二元组比较：

```
(number[j], j) < (number[i], i)
```

也就是号码小者优先；号码相同则进程号小者优先。

2. 基本实现

共享变量：

```
boolean choosing[n] = {false, false, ..., false};  
int number[n] = {0, 0, ..., 0};
```

进程 P_i 的算法：

```
while (true) {  
    choosing[i] = true;  
    number[i] = 1 + max(number[0], number[1], ..., number[n - 1]);  
    choosing[i] = false;  
  
    for (j = 0; j < n; j++) {  
        if (j == i) {  
            continue;  
        }  
  
        while (choosing[j]) {  
            ; // 等待  $P_j$  取号完成  
        }  
  
        while (number[j] != 0 &&  
              (number[j] < number[i] ||
```

```

        (number[j] == number[i] && j < i))) {
            ; // 等待优先级更高的 Pj
        }
    }

    临界区;

    number[i] = 0;

    剩余区;
}

```

3. 详细解释

choosing[i] 用来处理“正在取号”的中间状态。因为取号不是瞬间完成的，如果一个进程正在计算最大号码，另一个进程就直接比较 number，可能看到不稳定状态。等待 choosing[j] == false，表示 Pj 已经完成取号，比较结果才有意义。

number[i] 表示排队号码。号码越小，优先级越高；号码为 0 表示该进程不参与竞争。

等待条件：

```

number[j] != 0 &&
(number[j] < number[i] || (number[j] == number[i] && j < i))

```

表示：如果 Pj 正在排队，并且 Pj 的号码更小，或者号码相同但 Pj 的进程号更小，则 Pi 必须等待 Pj。

退出临界区时，Pi 把 number[i] 置为 0，表示不再排队，后面的进程可以继续比较并进入。

4. 性质总结

1. 可解决 n 个进程互斥。
2. 保证互斥：不同进程之间按 (number, process_id) 形成全序。
3. 保证先来先服务的近似公平性：先取到较小号码者优先。
4. 仍然采用忙等方式。
5. 不属于信号量算法。

5. 本类算法例题

题目

有 4 个进程 P0、P1、P2、P3 使用 Lamport 面包店算法。某时刻取号完成后：

```

number[0] = 3
number[1] = 0

```

```
number[2] = 2
number[3] = 3
```

问：当前正在竞争进入临界区的进程有哪些？它们进入临界区的先后顺序是什么？

解析

`number[i] == 0` 表示该进程不参与竞争，所以 `P1` 不在等待进入临界区。

参与竞争的是：

```
P0: (3, 0)
P2: (2, 2)
P3: (3, 3)
```

按 `(number, process_id)` 从小到大排序：

```
(2, 2) < (3, 0) < (3, 3)
```

所以进入临界区的先后顺序为：

```
P2 -> P0 -> P3
```

其中 `P0` 和 `P3` 号码相同，按进程号决定，`P0` 优先于 `P3`。

五、Eisenberg/McGuire 算法

Eisenberg/McGuire 算法用于 n 个进程的互斥。它使用每个进程的状态和一个共享的 `turn` 指针来控制进入临界区的顺序。

1. 三种状态

每个进程有一个状态 `flag[i]`，可能取以下三个值：

1. `idle`：进程不想进入临界区。
2. `want_in`：进程想进入临界区，正在争取进入资格。
3. `in_cs`：进程正在进入或已经进入临界区。

共享变量：

```
enum State { idle, want_in, in_cs };
State flag[n] = {idle, idle, ..., idle};
int turn = 0;
```

`turn` 指向当前优先检查的位置，用来减少饥饿并组织进程之间的竞争次序。

2. 基本实现

进程 P_i 的算法可写为：

```
while (true) {
    repeat {
        flag[i] = want_in;
        j = turn;

        while (j != i) {
            if (flag[j] != idle) {
                j = turn;
            } else {
                j = (j + 1) % n;
            }
        }

        flag[i] = in_cs;

        j = 0;
        while (j < n && (j == i || flag[j] != in_cs)) {
            j = j + 1;
        }
    } until (j >= n && (turn == i || flag[turn] == idle));

    turn = i;

    临界区;

    j = (turn + 1) % n;
    while (flag[j] == idle) {
        j = (j + 1) % n;
    }
    turn = j;

    flag[i] = idle;

    剩余区;
}
```

3. 详细解释

算法进入临界区前大致分为三个阶段。

第一阶段：声明意愿。

```
flag[i] = want_in;
```

这表示 `Pi` 想进入临界区，但还没有获得最终资格。

第二阶段：按照 `turn` 开始扫描。

```
j = turn;
while (j != i) {
    if (flag[j] != idle) {
        j = turn;
    } else {
        j = (j + 1) % n;
    }
}
```

这一段的作用是观察从 `turn` 开始到自己之前是否存在其他非空闲进程。如果遇到非空闲进程，就回到 `turn` 重新检查；如果某个进程空闲，就继续向后扫描。只有扫描到自己时，才说明自己有机会继续竞争。

第三阶段：尝试进入。

```
flag[i] = in_cs;
```

`Pi` 把自己状态改为 `in_cs`。但这还不是直接进入临界区，因为可能有其他进程也刚刚把状态改成 `in_cs`。因此还要检查是否存在其他 `flag[j] == in_cs`。

```
j = 0;
while (j < n && (j == i || flag[j] != in_cs)) {
    j = j + 1;
}
```

如果找到其他进程也处于 `in_cs`，说明竞争还没有解决，必须重新尝试。如果没有找到其他 `in_cs` 进程，并且满足：

```
turn == i || flag[turn] == idle
```

则说明当前进程可以获得进入资格。随后执行：

```
turn = i;
临界区;
```

退出临界区时，从 `turn + 1` 开始寻找下一个非空闲进程，把 `turn` 交给它：

```
j = (turn + 1) % n;
while (flag[j] == idle) {
    j = (j + 1) % n;
}
turn = j;
flag[i] = idle;
```

这样可以在多个等待者之间形成轮转，减少某个进程长期得不到机会的可能。

4. 性质总结

- 1. 可解决 n 个进程互斥。
- 2. 使用三态标志，比单纯 true/false 更细致地区分“想进入”和“正在进入临界区”。
- 3. 使用 turn 指针安排竞争顺序，帮助实现有限等待。
- 4. 仍然属于软件互斥算法，不依赖信号量。
- 5. 算法较复杂，考试通常重在理解状态含义、turn 作用和进入条件。

5. 本类算法例题

题目

在 Eisenberg/McGuire 算法中，flag[i] 有 idle、want_in、in_cs 三种状态。请说明这三种状态分别表示什么，并解释为什么算法需要 turn 指针。

解析

idle 表示进程当前不想进入临界区，也不参与竞争。

want_in 表示进程想进入临界区，已经提出申请，但还没有获得进入资格。

in_cs 表示进程正在尝试进入或已经进入临界区。算法会检查是否存在其他进程也处于 in_cs，以避免多个进程同时进入临界区。

turn 指针的作用是组织多个进程之间的竞争顺序。若没有 turn，多个进程可能反复同时竞争，难以保证等待者之间的公平性。turn 使算法可以从某个指定位置开始扫描，并在退出临界区时把机会交给下一个非空闲进程，从而帮助实现有限等待，减少饥饿。

六、横向对比与记忆表

算法类别	适用进程数	核心变量	是否保证互斥	主要问题或特点
单标志法	2	turn	是	严格轮换，违反空闲让进

算法类别	适用进程数	核心变量	是否保证互斥	主要问题或特点
双标志先检查法	2	flag[2]	否	可能两个进程同时进入
双标志后检查法	2	flag[2]	是	可能死锁
Dekker 算法	2	flag[2]、turn	是	正确但较复杂，忙等
Peterson 算法	2	flag[2]、turn	是	简洁经典，只适用于两个进程
Lamport 面包店算法	n	choosing[n]、number[n]	是	取号排队，号码小者优先
Eisenberg/McGuire 算法	n	flag[n]、turn	是	三态标志加 turn，算法较复杂

七、常见考点提醒

1. 单标志法的典型错误不是破坏互斥，而是违反空闲让进。
2. 双标志先检查法的典型错误是可能同时进入临界区。
3. 双标志后检查法的典型错误是可能死锁。
4. Peterson 算法中 `turn = j` 的含义是发生竞争时让对方优先。
5. Dekker 算法中冲突时会临时撤销自己的申请，体现主动让步。
6. 面包店算法按（号码，进程号）排序，`number[i] == 0` 表示不竞争。
7. 面包店算法不是信号量算法，不能把取号误写成 P/V 操作。
8. Eisenberg/McGuire 算法要记住三种状态：`idle`、`want_in`、`in_cs`，以及 `turn` 的轮转作用。